

ENSIL-ENSCI — Université de Limoges
Projet de fin d'études — MIX 5 — Promotion 2026

Projet V.E.S.P.A.

Tourelle optronique de suivi prédictif par stéréo-vision
à neutralisation laser dynamique — prototype mk.2

Rapport technique

Guide de reproduction — matériel, système, firmware

Équipe

Antonin Delmas
Clément Desmedt
Clément Gilson

Encadrement

Michel Ousset — encadrant
Thierry Cortier — enseignant

Document de clôture — juillet 2026
Tout ce qui est nécessaire pour reconstruire le prototype mk.2 depuis zéro.

Table des matières

1	Objet de ce document	2
2	Architecture matérielle	2
3	Nomenclature (BOM)	2
4	Câblage	3
5	Installation du Jetson Orin Nano	5
5.1	Système de base	5
5.2	Overlays d'arbre de périphériques	5
5.3	Vérifications	5
5.4	Bus CAN virtuel (indispensable pour travailler)	6
6	Compilation	7
6.1	Nœud de vision (Jetson)	7
6.2	Nœud de mouvement (STM32)	7
7	Calibration	8
7.1	Ordre des opérations	8
8	Mise en route	8
9	Zone opérationnelle et dimensionnement	10
10	Structure du dépôt	10
11	Synthèse de l'état livré	11

1 Objet de ce document

Ce document est un **guide de reproduction**. Il décrit, dans l'ordre où il faut les exécuter, les opérations permettant de reconstituer le prototype V.E.S.P.A. mk.2 : approvisionnement, câblage, installation système, compilation des deux firmwares, calibration, mise en route.

Il ne justifie pas les choix de conception — cela relève du rapport de soutenance (`docs/Rapport_PFE_2026_VESPA`). Il ne raconte pas non plus les impasses — cela relève de `docs/Difficultes_techniques.pdf`.

Lisez d'abord « Difficultés techniques »

Plusieurs étapes de ce guide comportent des pièges qui **ne se voient pas** et dont le symptôme n'a aucun rapport apparent avec la cause (une broche qui coupe la console série, un outil NVIDIA qui détruit la configuration des caméras). Elles sont signalées ici, mais elles ne sont *expliquées* que dans le document des difficultés. Le temps passé à le lire sera regagné plusieurs fois.

État de ce qui est reproductible. Ce guide mène à un système qui **suit une cible ArUco en 3D et oriente la tourelle**. Il ne mène pas à un système qui reconnaît un frelon (modèle indisponible), ni qui tire (watchdog non implémenté, laser jamais mis sous tension). Voir la synthèse en fin de document.

2 Architecture matérielle

Nœud	Calculateur	Rôle
<code>vision_node</code>	Jetson Orin Nano Super 8 Go	Acquisition stéréo synchronisée, détection, triangulation, prédiction, émission de la consigne
<code>motion_node</code>	NUCLEO-G431RB	Asservissement FOC en boucle fermée des axes PAN et TILT
<code>coordination_node</code>	ESP32	Collimation, tir, watchdog — non implémenté

Les nœuds communiquent sur un **bus CAN 2.0B à 500 kbit/s** (trames standard 11 bits). Le dictionnaire de données complet est en annexe (`docs/annexes/protocole_can.md`) ; sa définition normative est `software/common/can_protocol.h`.

3 Nomenclature (BOM)

Nomenclature du prototype mk.2, telle qu'effectivement approvisionnée. Total $\approx$ **830 € TTC**.

Élément	Qté	Prix TTC
<i>Calcul et électronique</i>		
Jetson Orin Nano Super Developer Kit	1	311,18 €
B-G431B-ESC1 (<i>voir note ci-dessous</i>)	2	16,31 €
ESP32-C3 SuperMini (<i>coordination — non utilisé</i>)	1	4,29 €
Alimentation AC-DC 24 V / 150 W	1	5,29 €
Transceivers CAN (lot de 10, type WCMCU-230 / SN65HVD230)	1	5,19 €
TMC2209 (lot de 4) (<i>collimation</i>)	1	9,09 €
<i>Vision</i>		
Module caméra Arducam OV9281 1 MP monochrome <i>global shutter</i>	2	142,59 €
Objectif M12 5 MP 4,6 mm sans distorsion	2	9,29 €
<i>Mécanique et actionneurs</i>		
iFlight iPower GM5208-12 + encodeur AS5048A (paire)	1	134,99 €
Actionneur linéaire course 35 mm	1	20,42 €
Actionneur linéaire course 10 mm	2	1,99 €
Capteur photoélectrique en U (<i>prise d'origine</i>)	1	2,54 €
<i>Optique de tir</i>		
Diode laser bleue 5,5 W (Classe 4)	1	83,69 €
Lentille asphérique K9 D63 F100 (<i>L2, focale 100 mm</i>)	1	23,59 €
<i>Sécurité — non optionnel</i>		
Lunettes de protection OD7+ CE 190–550 nm	1	30,29 €

Deux divergences entre la nomenclature et le matériel réellement utilisé

1. **Cartes de commande moteur.** La nomenclature liste des **B-G431B-ESC1**. Le firmware livré cible en réalité une **NUCLEO-G431RB** portant deux shields **X-NUCLEO-IHM16M1** (STSPIN830) — c'est ce que déclarent `platformio.ini` (`board = nucleo_g431rb`) et `pinout.h`. **Suivre le code, pas la nomenclature.**
2. **Le laser.** La nomenclature (et donc le matériel acheté) porte sur une diode **5,5 W bleue**. Le dimensionnement théorique du rapport de soutenance a été conduit pour $P = 5 \text{ W}$ à 460 nm. Les deux valeurs sont cohérentes en ordre de grandeur ; c'est la diode 5,5 W qui est physiquement présente. **Classe 4 dans les deux cas.**

Moteurs. GM5208-12, PMSM *pancake* : **7 paires de pôles** (14 aimants), $R_s = 7,48 \Omega$, $L_s = 4,49 \text{ mH}$, tension nominale 24 V, courant nominal 1,5 A. Encodeur magnétique absolu AS5048A, **14 bits** $\rightarrow$ résolution $360/2^{14} \approx 0,022^\circ$, borne théorique de précision de la tourelle.

4 Câblage

Le câblage complet, broche à broche, est en annexe : `docs/annexes/cablage.md`. Il n'est pas reproduit ici, mais **deux points sont critiques** et doivent être connus avant de souder :

Les deux pièges du câblage

1. **Le potentiomètre pan n'est PAS sur PA2, mais sur PB14.**
PA2 est câblée sur `USART2_TX` (console série du ST-LINK) sur la NUCLEO-G431RB. Y brancher le potentiomètre **coupe la console**. Le fichier historique `STM32_WiringManifest.txt` indique encore **PA2** : il est faux.

2. PA4 est réservée au SPI (NSS matériel), pas au retour de courant.

Elle doit rester haute, sous peine de faire basculer le SPI1 — qui porte les deux encodeurs — en mode esclave. Le retour de courant de la phase W du moteur 1 est sacrifié en conséquence.

Bus CAN. Transceiver **3,3 V uniquement** (SN65HVD230 / WCMCU-230). **Ne jamais l'alimenter en 5 V** : cela détruit le composant et peut endommager les broches du Jetson.

- Côté Jetson : CTX → broche **31**, CRX → broche **29**, 3V3 → broche 1, GND → broche 6.
- Côté STM32 : CAN_TX = PB9, CAN_RX = PB8.
- **Masse commune obligatoire** entre les nœuds (3^e fil) : le CAN est différentiel, mais le transceiver a une limite de tension de mode commun.
- Terminaison **120 Ω** aux *deux extrémités* du bus, et *seulement* aux extrémités. Les modules WCMCU-230 embarquent une résistance (sérigraphiée R2, marquage « 121 ») : la dessouder sur tout nœud qui n'est pas en bout de bus.

5 Installation du Jetson Orin Nano

L'étape la plus délicate du projet

Le Jetson ne fonctionne **pas** avec le noyau JetPack standard : les caméras OV9281 exigent le **noyau Arducam**. Et une fois cette configuration en place, l'outil `jetson-io.py` **ne doit plus jamais être exécuté** — il régénère globalement les *overlays* d'arbre de périphériques et **détruit la synchronisation stéréo**.

5.1 Système de base

1. Flasher le **JetPack 6.2** (L4T r36.4) sur le Jetson Orin Nano Super, via NVIDIA SDK Manager. Racine sur NVMe recommandée (c'est la configuration d'origine).
2. Installer le **noyau et les pilotes Arducam** pour l'OV9281, en suivant la procédure du constructeur. Cela installe un noyau dédié dans `/boot/arducam/`.

5.2 Overlays d'arbre de périphériques

La configuration matérielle est portée par trois *overlays* chargés au démarrage. Les fichiers de référence sont conservés dans `jetson/` :

Overlay	Rôle
<code>...-arducam-dual.dtbo</code>	Double caméra OV9281. C'est lui qui résout le conflit apparent d'adresse I ² C (un bus par port CSI).
<code>jetson-io-hdr40-user-custom.dtbo</code>	PWM de synchronisation (broches 32 et 33 de l'en-tête 40 points). Ne pas modifier ni régénérer.
<code>can0-pinmux.dtbo</code>	Tentative de routage du CAN0. Ne fonctionne pas — voir « Difficultés techniques ».

Le fichier `/boot/extlinux/extlinux.conf` de référence est fourni (`jetson/extlinux.conf.reference`). L'entrée active y charge le noyau Arducam et la liste d'*overlays* :

```

LABEL JetsonIO
    MENU LABEL Custom Header Config: <CSI Camera ARDUCAM Dual>
    LINUX /boot/arducam/Image
    FDT /boot/arducam/dts/dtb/tegra234-p3768-0000+p3767-0005-nv-super.dtb
    INITRD /boot/initrd
    APPEND ${cbootargs} root=PARTUUID=... rw rootwait rootfstype=ext4 ...
    OVERLAYS /boot/arducam/dts/tegra234-p3767-camera-p3768-arducam-dual.dtbo ,
             /boot/jetson-io-hdr40-user-custom.dtbo ,
             /boot/can0-pinmux.dtbo

```

5.3 Vérifications

```

# Les deux cameras doivent apparaitre
ls /dev/video*                # -> /dev/video0 /dev/video1
v4l2-ctl --list-devices

# Le PWM de synchronisation doit etre exportable
ls /sys/class/pwm/pwmchip3/

# Le CAN physique : NE FONCTIONNE PAS a ce jour (verrou ouvert)
sudo bash -c "cat /sys/kernel/debug/pinctrl/*/pinmux-pins | grep -i paa"
# -> aucune sortie = les broches PAA ne sont pas reclamées

```

5.4 Bus CAN virtuel (indispensable pour travailler)

Le bus physique étant inopérant, le développement et la validation se font sur `vcan0`. Le fichier `jetson/systemd/vcan.conf` charge le module au démarrage :

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
candump vcan0                                # observer les trames
```

Le service `jetson/systemd/can0-setup.service` configure le bus *physique* (500 kbit/s) — il est écrit pour ne se déclencher que si le noyau crée réellement le périphérique `can0`, ce qui n'est jamais arrivé.

6 Compilation

6.1 Nœud de vision (Jetson)

```
sudo apt install libopencv-dev libi2c-dev libzmq3-dev cppzmq-dev \
v4l-utils can-utils cmake build-essential
```

Dépendances exigées mais non utilisées

Le CMakeLists.txt déclare `find_package(CUDA REQUIRED)` et `find_package(VPI 3.0 REQUIRED)`, ainsi qu'un test sur `i2c/smbus.h`. **Le chemin de code actuel (ArUco + OpenCV) n'en a pas besoin** : ces dépendances sont un vestige de la voie TensorRT abandonnée. Elles restent *obligatoires* pour que CMake aboutisse. Si la reprise ne vise pas l'inférence, ces trois exigences peuvent être retirées sans dommage.

```
cd software/vision_node
mkdir -p build && cd build
cmake .. && make -j6
```

Cibles produites.

Binaire	Usage
vision_node	Cible de production. Sans interface, sans ZeroMQ. Émet sur le CAN.
stereo_tracker	Idem + flux ZeroMQ pour la visualisation (<i>live viewport</i>).
calibration_bridge	Pont ZeroMQ pour les outils Python de calibration.
test_camera_hal	Test unitaire d'acquisition.
test_hardware_sync	Test de la synchronisation stéréo — à lancer en premier.
test_stereo_pipeline	Test de la triangulation hors ligne.

Exécution. Les droits *root* sont nécessaires (accès à `/sys/class/pwm` et `/dev/video*`) :

```
sudo ./vision_node
```

6.2 Nœud de mouvement (STM32)

Projet PlatformIO. La bibliothèque commune est liée par lien symbolique, ce qui garantit qu'il n'existe **qu'une seule définition** du protocole CAN :

```
cd software/motion_node
pio run                # compiler
pio run -t upload      # televerser (sonde ST-LINK integree a la Nucleo)
pio device monitor -b 115200
```

Configuration (`platformio.ini`) : `board = nucleo_g431rb`, framework Arduino, SimpleFOC 2.3.2, avec activation explicite des modules HAL OPAMP et FDCAN.

Ne jamais instancier MotionCan sans arguments

Le constructeur déclare des valeurs par défaut (PA11, PA12) qui sont **fausses** pour ce câblage — **PA11** est le EN/FAULT du moteur 1. Toujours passer les broches explicitement : `MotionCan canBus(CAN_RX, CAN_TX)`; soit **PB8/PB9**.

7 Calibration

Trois outils web (Flask) tournent sur le Jetson et s'utilisent depuis un navigateur distant — la machine étant sans écran.

Un seul programme à la fois sur le matériel

Le nœud de vision et les outils Python se disputent le bus I²C. Un verrou d'exclusion (/tmp/vespa_hardware.lock) fait **échouer volontairement** le second démarrage. **Ce n'est pas un bug** : arrêter l'un avant de lancer l'autre.

7.1 Ordre des opérations

1. **Exposition et gain** — software/tools/calib_exposition/

```
cd software/tools/calib_exposition && ./run_exposure.sh # http://<
jetson>:5000
```

Régler l'exposition et le gain jusqu'à obtenir un marqueur net et contrasté. Les profils sont enregistrés dans software/vision_node/config/profiles/ (P1, P2 « sombre », P3 « lumineux »), le profil actif étant désigné par config/.active_profile.

Limite connue

Le capteur n'applique pas le même calcul de temps-ligne en mode maître et en mode déclenchement externe. **Les valeurs réglées par cet outil ne correspondent pas exactement à celles du pipeline C++**. Prévoir un ajustement final en conditions réelles.

2. **Calibration stéréo** — software/tools/calib_stereo/

```
cd software/tools/calib_stereo && ./run_calibration.sh # http://<
jetson>:5000
```

Méthode de Zhang [2] sur mire d'échiquier. **Couvrir tout le champ** — coins, bords, près et loin, et pas seulement le centre : sans couverture des bords, la distorsion ne peut pas être résolue, la focale « hallucine » et l'erreur RMS explose. Résultat écrit dans config/stereovision_settings.json.

3. **Vérification** — software/tools/viewport_live/

```
cd software/tools/viewport_live && ./run_liveviewport.sh
```

Affiche le flux avec superposition : position courante du marqueur et **visée prédictive** (réticule cyan) issue du filtre de Kalman.

8 Mise en route

1. **Vérifier la synchronisation** avant toute chose :

```
sudo ./build/test_hardware_sync
```

La dérive inter-caméras doit être nulle. Si elle ne l'est pas, ne pas aller plus loin : la triangulation serait fautive.

2. **Monter le bus** (virtuel, en l'état actuel) : vcan0, et lancer candump sur une seconde console.
3. **Démarrer le nœud de mouvement**. À la mise sous tension, la séquence FOC s'initialise. Les décalages d'encodeur sont **codés en dur** dans main.cpp : motorPan.sensor_offset = 4,7500, motorTilt.sensor_offset = 4,8179 — ils repoussent le passage 0/2 π hors

de la plage utile. **Ils sont propres à un montage mécanique donné** et devront être réétalonnés sur un nouveau montage (`Examples/OneTimeInitialisationAngle.cpp`).

4. **Démarrer le nœud de vision** : `sudo ./vision_node`.
5. **Présenter le marqueur ArUco** dans la *killbox*. La tourelle doit suivre.

Butées logicielles (`main.cpp`, en radians) — elles protègent la mécanique et doivent être revues sur un nouveau montage :

Axe	Min	Max
PAN	$-0,52 \text{ rad } (-30^\circ)$	$+0,52 \text{ rad } (+30^\circ)$
TILT	$-0,26 \text{ rad } (-15^\circ)$	$+0,78 \text{ rad } (+45^\circ)$

Arrêt d'urgence. Bouton bleu (**PC13**). L'arrêt est **latché** : coupure des deux étages de puissance, diffusion d'un *heartbeat* **FAULT**, puis boucle infinie. **Seul un reset matériel (bouton noir) permet de repartir.** C'est délibéré : sur un système portant un laser de Classe 4, aucun redémarrage automatique n'est acceptable.

9 Zone opérationnelle et dimensionnement

La killbox. Volume dans lequel une cible peut être neutralisée : parallélépipède de **500 mm × 310 mm × 300 mm** (longueur × largeur × hauteur). La tourelle est placée **en surplomb**, de sorte que le tir soit dirigé vers le sol.

Optique. Le compromis porte sur le GSD (*Ground Sampling Distance*) : il ne doit pas dépasser $\approx 0,5 \text{ mm/px}$ pour rester exploitable par un modèle de détection. Pour un capteur 1280×800 , le script `simulation/vision/Camera_PosResAttitude_v6.m` donne :

$$f = 6 \text{ mm}, \quad d_n = 990 \text{ mm}$$

Cadence. La boucle complète doit tenir dans **16,6 ms** (60 Hz). À 60 fps, le système suit une cible se déplaçant jusqu'à $0,6 \text{ m} \cdot \text{s}^{-1}$ — ce qui suppose un **frelon en vol de guet quasi stationnaire**, comportement effectivement documenté devant les ruches [1]. Un frelon en vol rapide ($\approx 6 \text{ m} \cdot \text{s}^{-1}$) exigerait 600 fps : **hors de portée**, et c'est une limite assumée du système.

Obturateur global : pourquoi il est non négociable. Le flou de mouvement vaut $b = v \cdot t_{\text{exp}}/R$. À $0,6 \text{ m} \cdot \text{s}^{-1}$, avec 2 ms d'exposition et $R = 0,5 \text{ mm/px}$, on obtient $b = 2 \text{ px}$ — sur une cible qui n'en mesure que 20 à 35. Un obturateur à balayage (*rolling shutter*) est donc éliminé. Or aucun capteur **trichrome** à obturateur global n'existe à prix abordable : **d'où le capteur monochrome, et d'où l'incompatibilité avec VespAI** (voir « Difficultés techniques »).

10 Structure du dépôt

<code>project-vespa/</code>	
<code> - docs/</code>	Rapports (PDF + sources LaTeX), annexes, references
<code> - src/</code>	Sources LaTeX + vespa.bib + figures
<code> - annexes/</code>	Cablage, protocole CAN, BOM, setup Jetson, securite
<code> - references/</code>	Articles, datasheets, etudes preparatoires
<code> - hardware/</code>	Mecanique (impressions 3D), moteurs
<code> - jetson/</code>	Configuration systeme (overlays, services CAN,
<code> extlinux)</code>	
<code> - simulation/</code>	Scripts MATLAB de dimensionnement (vision + optique)
<code> - software/</code>	
<code> - common/</code>	Protocole CAN partage (C, portable STM32/ESP32/Linux
<code>)</code>	
<code> - vision_node/</code>	Jetson - C++17 / OpenCV / V4L2 / SocketCAN (CMake)
<code> - motion_node/</code>	STM32G431RB - SimpleFOC / FDCAN (PlatformIO)
<code> - coordination_node/</code>	ESP32 - NON IMPLEMENTE (specification seule)
<code> - tools/</code>	Calibration stereo, exposition, viewport
<code> - legacy_mk1_bench/</code>	Banc de caracterisation mk.1 (Arduino Uno)
<code> - tools/</code>	Scripts utilitaires du depot

Le banc mk.1. `software/legacy_mk1_bench/` contient le code Arduino ayant servi à caractériser le prototype de première génération (comparaison des algorithmes « classique » et « accumulateur », mesure du *jitter*, test d'endurance). Ce sont les **annexes A à D du rapport de soutenance**. Il est conservé parce qu'il porte les mesures qui justifient l'abandon des moteurs pas-à-pas.

11 Synthèse de l'état livré

Sous-système	État	Validé sur
Acquisition stéréo	Synchronisation matérielle 60 Hz, dérive 0,000 ms	matériel réel
Triangulation	Stéréo éparsé sur marqueur ArUco	matériel réel
Prédiction	Filtre de Kalman, compense la latence de la chaîne	matériel réel
Asservissement	FOC boucle fermée, 2 axes, butées, arrêt d'urgence	matériel réel
Protocole CAN	Sérialisation, priorités, <i>heartbeats</i>	vcan0 seulement
Bus CAN physique	Ne fonctionne pas — <i>pinmux</i> non réclamé	—
Détection du frelon	Absente — remplacée par ArUco	—
Watchdog	Non implémenté	—
Tir laser	Jamais effectué (décision de sécurité)	—

Avant toute mise sous tension du laser

Le laser est de **Classe 4**. Il n'a jamais été alimenté, faute de watchdog. Lire docs/annexes/securite_laser.md **en entier** avant d'y toucher. La collimation dynamique réduit le risque d'incendie, **pas** le risque oculaire, qui persiste sur près de 200 m.

Références

- [1] Adrien Perrard, Jean Haxaire, Agnès Rortais, and Claire Villemant. Observations on the colony activity of the Asian hornet *vespa velutina* Lepeletier 1836 (Hymenoptera : Vespidae : Vespinae) in France. *Annales de la Société Entomologique de France*, 45(1) :119–127, 2009. Cycle et activité de la colonie. Justifie le comportement de « vol stationnaire de guet » devant la ruche, hypothèse opérationnelle sur laquelle repose le compromis à 60 fps.
- [2] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11) :1330–1334, 2000. Méthode de calibration implémentée par `cv::calibrateCamera` / `cv::stereoCalibrate`, utilisée par l'outil `software/tools/calib_stereo`.